Singly Linked List – GetNthNodeFromEnd and Time Complexity.

This function helps us to get the last node from the end of linked list.

Program :

```
int getNthNodeFromEnd(Node **head, int node)
{
    int len = 0;
    Node *current;
    current = *head;
    while (current != nullptr)
    {
        current = current->next;
        len++;
    }
    if (node < 1 || node > len)
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
    else
    {
        current = *head;
        for (int i = 0; i < len - node; i++)
        {
            current = current->next;
        }
        return current->data;
    }
}
```

```
 case 20:
        {
            cout << endl;
            cout << endl;
            cout << "Nth node from end, where 1 is the last node (1 based
indexing):" << endl;
            int n;
            cout << "Enter the value of node: ";
            cin >> n;
            cout << "The " << n << "th node from the end is: " <<
getNthNodeFromEnd(&head, n) << endl;
            cout << endl;
            cout << endl;
            break;
        }
```

Program Analysis:

Function Call:

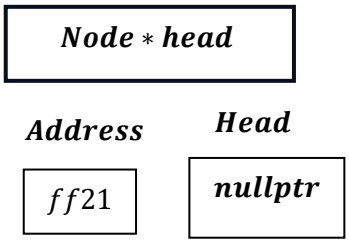
$$\textit{Function call occurs when Case 20 is initiated from Switch} - \textit{Case}:$$

$$\textit{cout} \ldots\ldots \textit{getNthNodeFromEnd}(\&\textit{head}, \textit{n});$$

$$\textit{There are two criteria of input Linked List}:$$

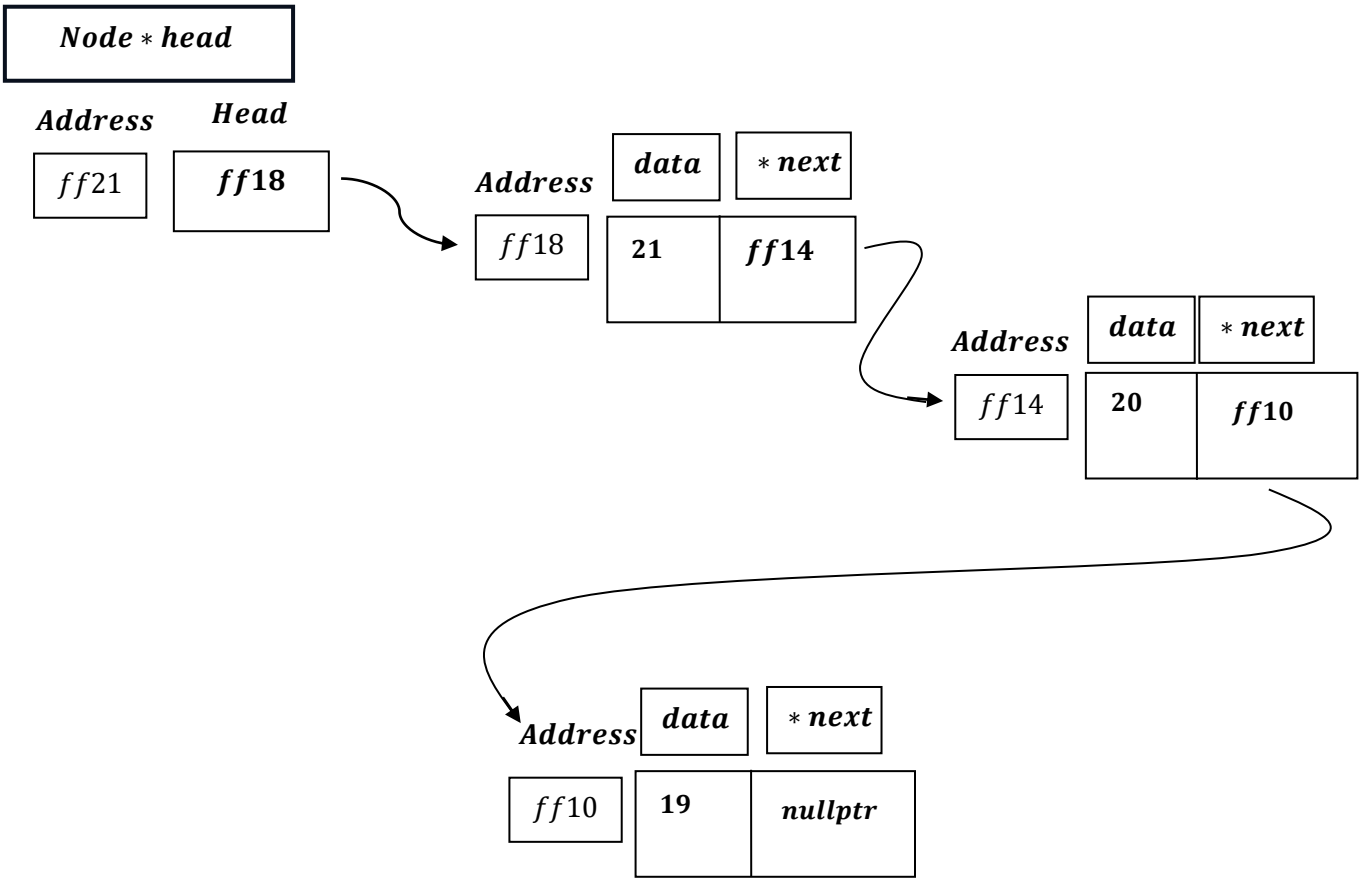
1) *Linked List is Empty*:

Pictorially depicted as:

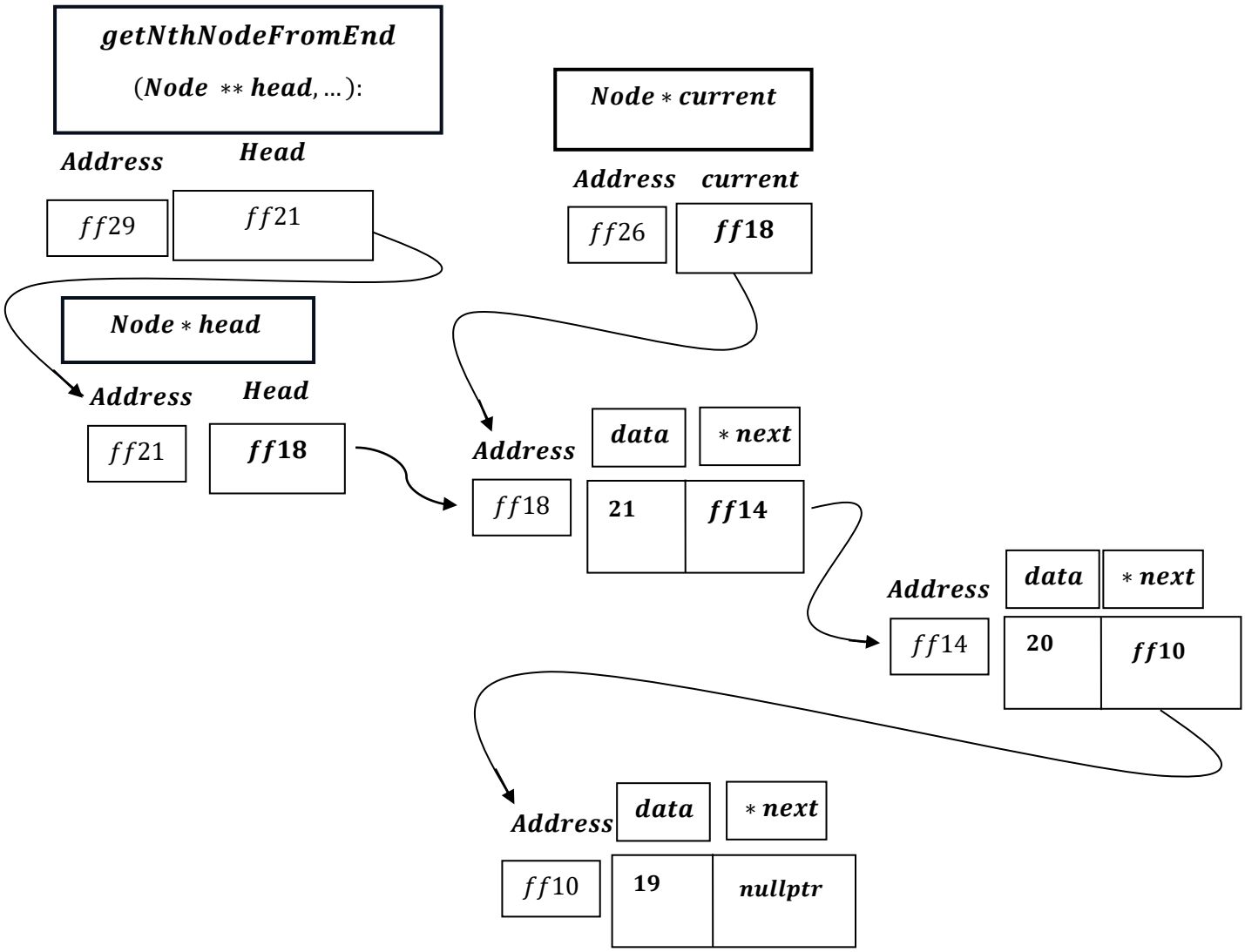


2) *Linked List is not Empty*:

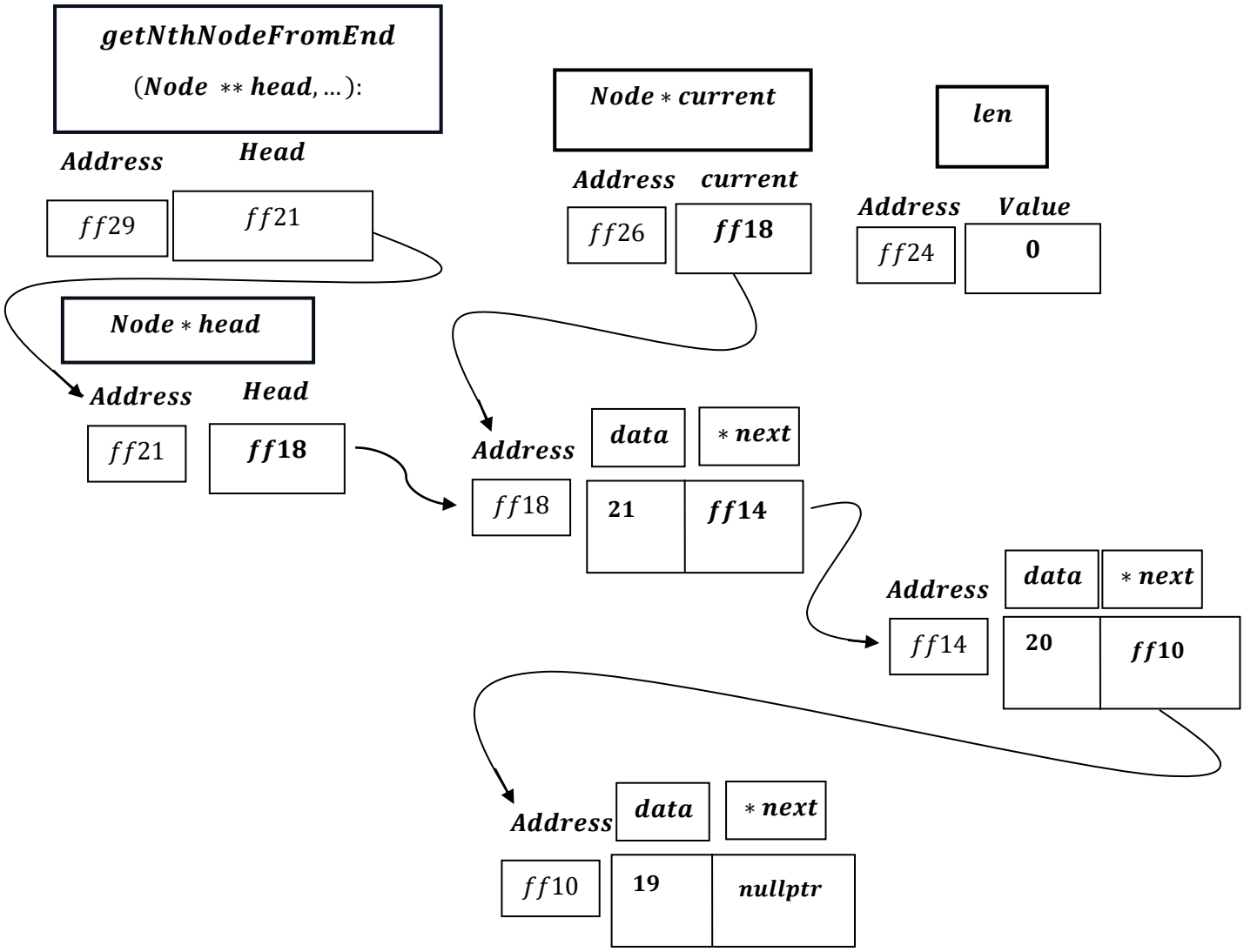
Pictorially depicted as:



```
Node *current;
current = *head;
```



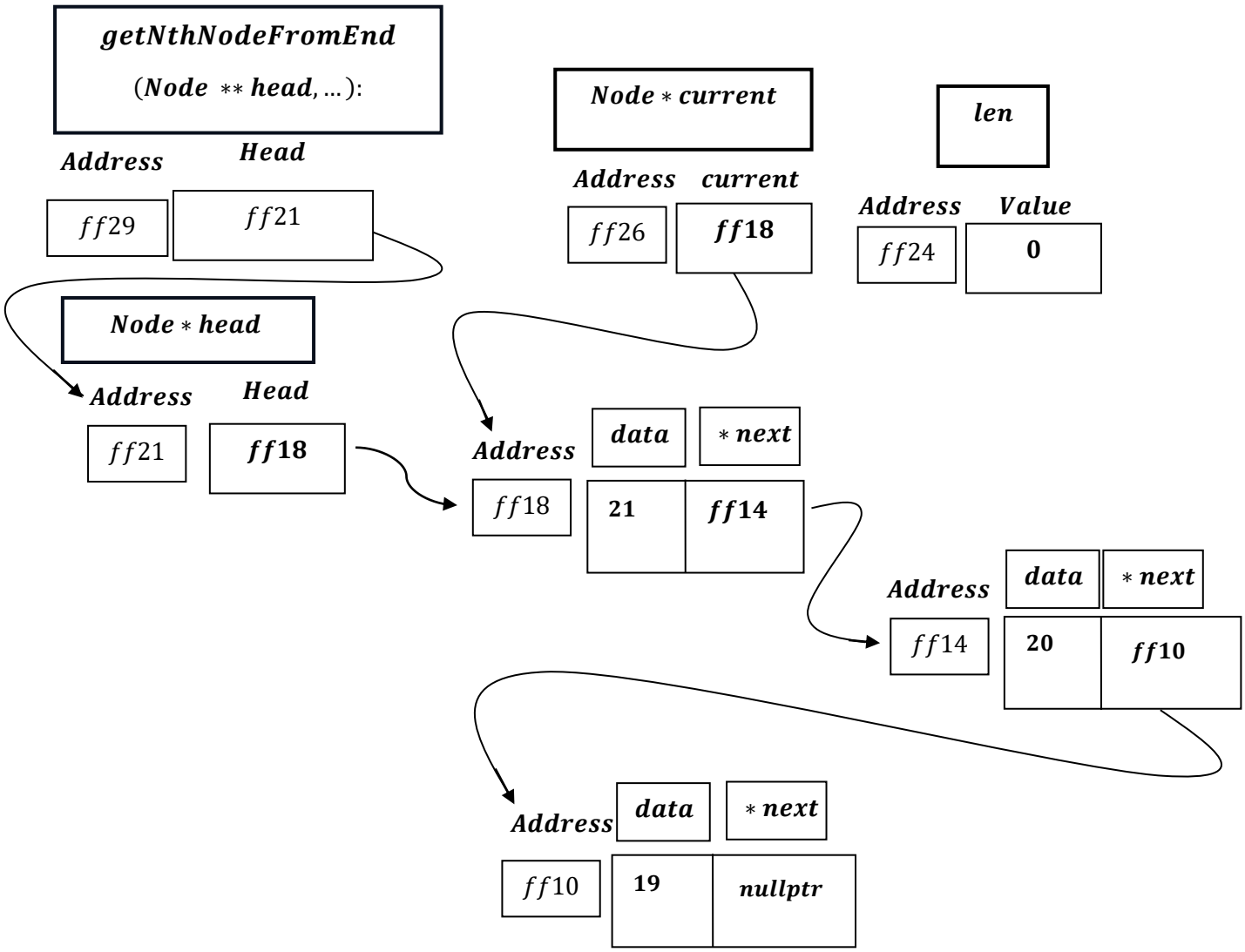

```
    int len = 0;
```



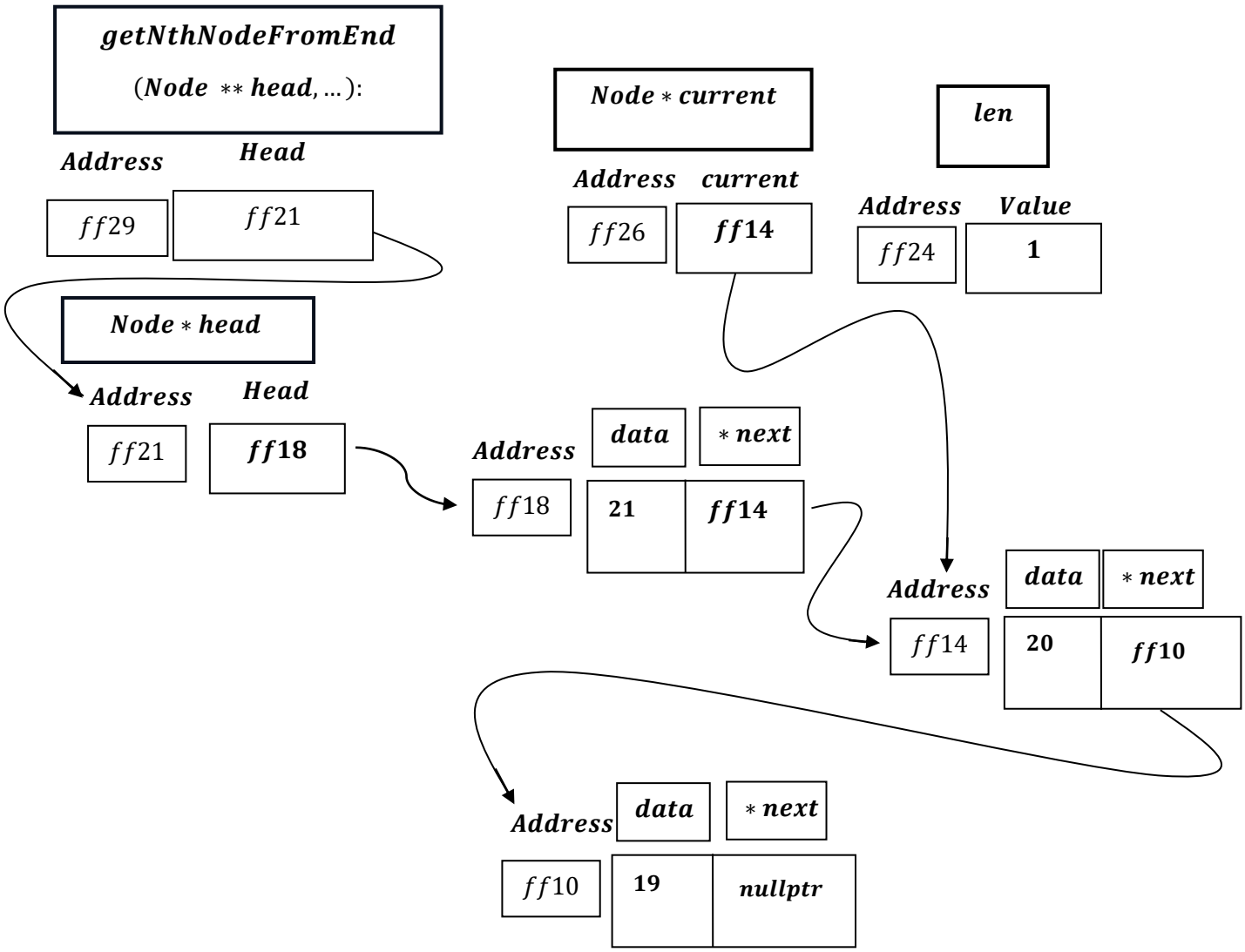
***Now , we will count length of the linked list*: –**

```
    while (current != nullptr)
    {
        current = current->next;
        len++;
    }
```

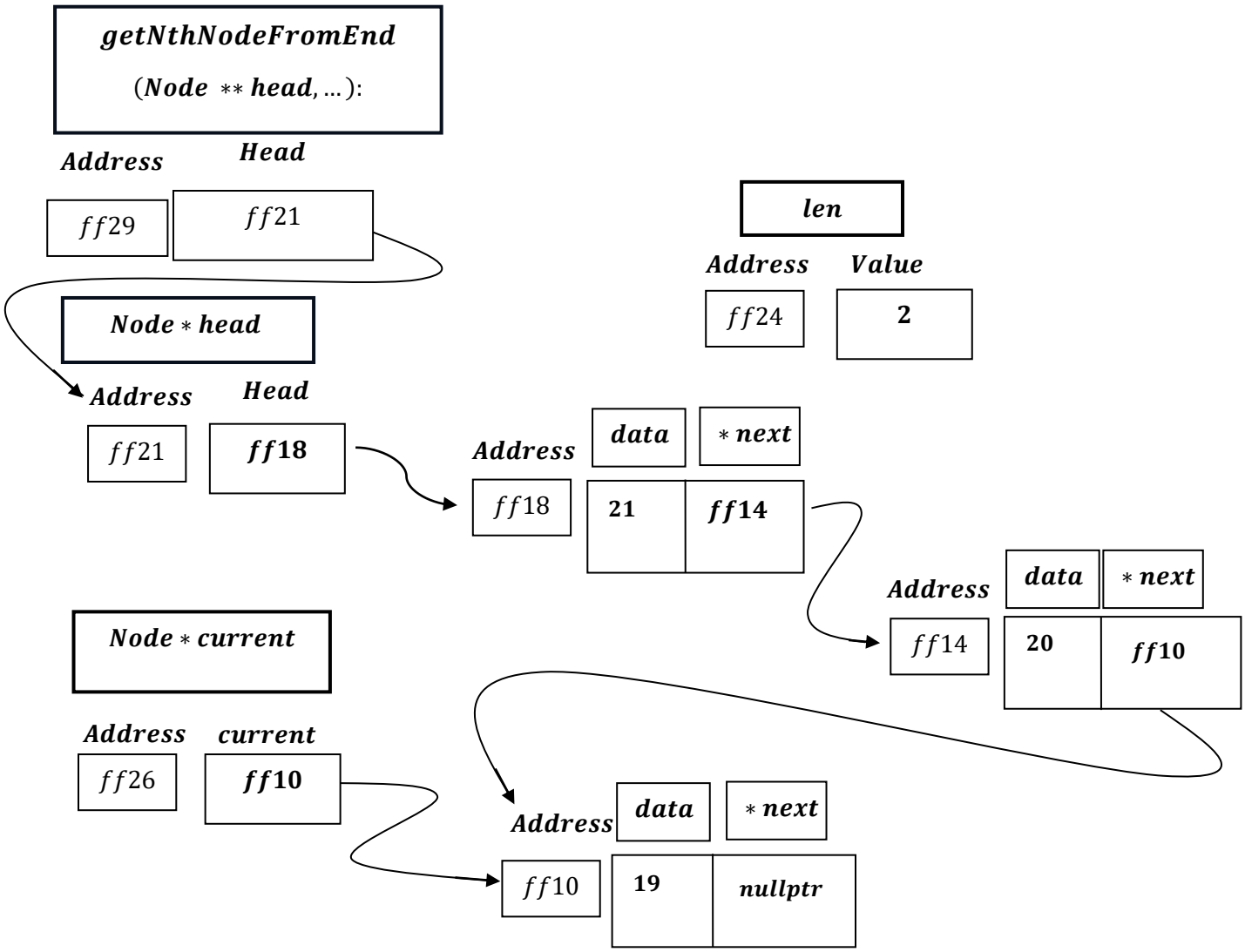
***We just recall the above picture* :**



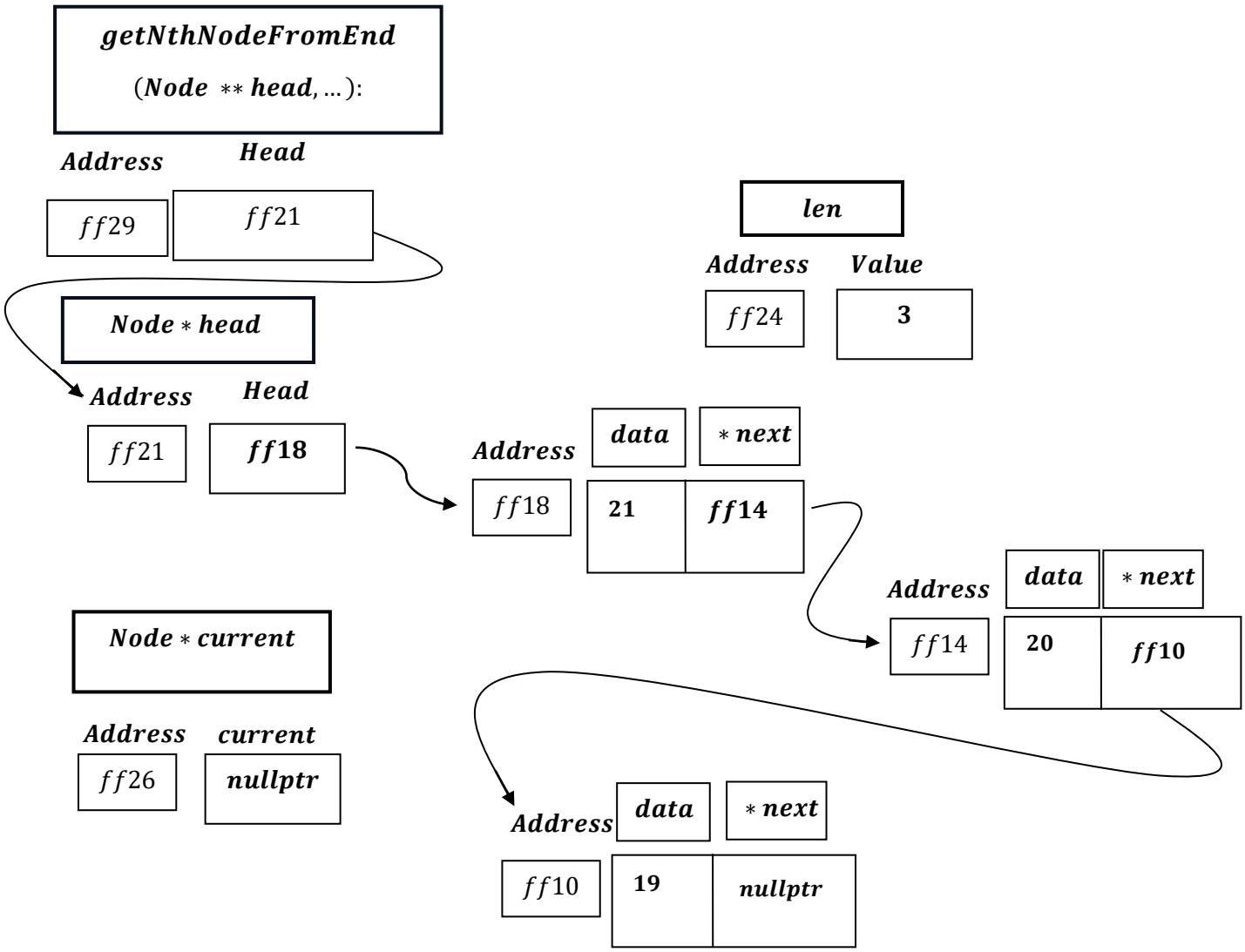

```
while(current != nullptr){
        current = current → next; i.e. current = ff14;
        len++; i.e. len = 0 + 1 = 1;
}
```




```
while(current != nullptr){
        current = current → next; i.e. current = ff10;
        len ++; i.e. len = 1 + 1 = 2;
}
```




```
while(current != nullptr){
        current = current → next; i.e. current = nullptr;
        len ++; i.e. len = 2 + 1 = 3;
}
```



Now, it checks whether user input `node` is less than `1` or `node` greater than `lengthOfLinkedList` as shown below:

```
    if (node < 1 || node > len)
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
```

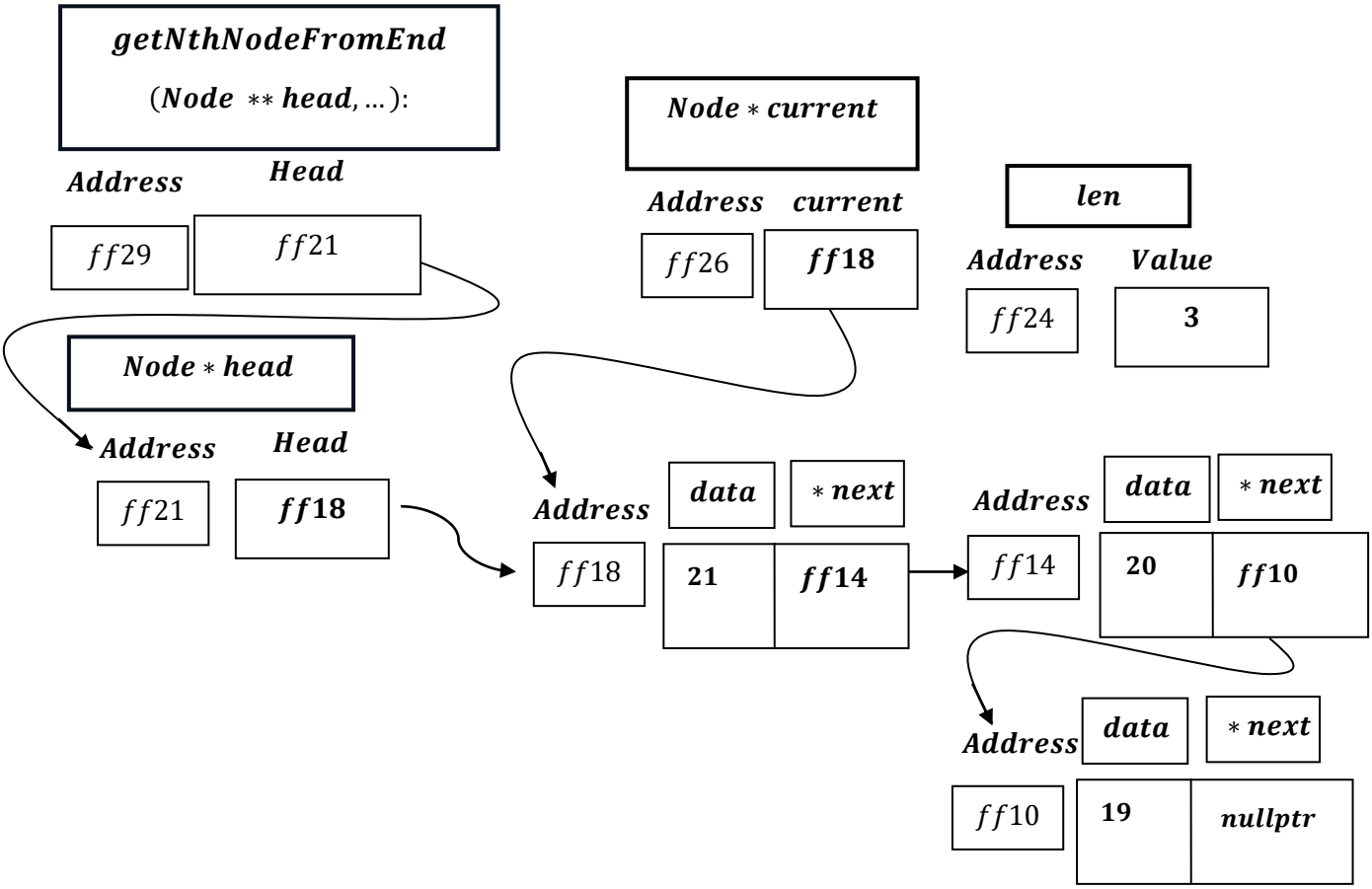
It outputs ``Error: Invalid index.`` and returns `−1`.

Now if user input node not less than 1 or `node` not greater than `lengthOfLinkedList` , then it will move to else part:

```
    else
    {
        current = *head;
        for (int i = 0; i < len - node; i++)
        {
            current = current->next;
        }
        return current->data;
    }
}
```

Now,

```
current = *head;
```



$Now,$

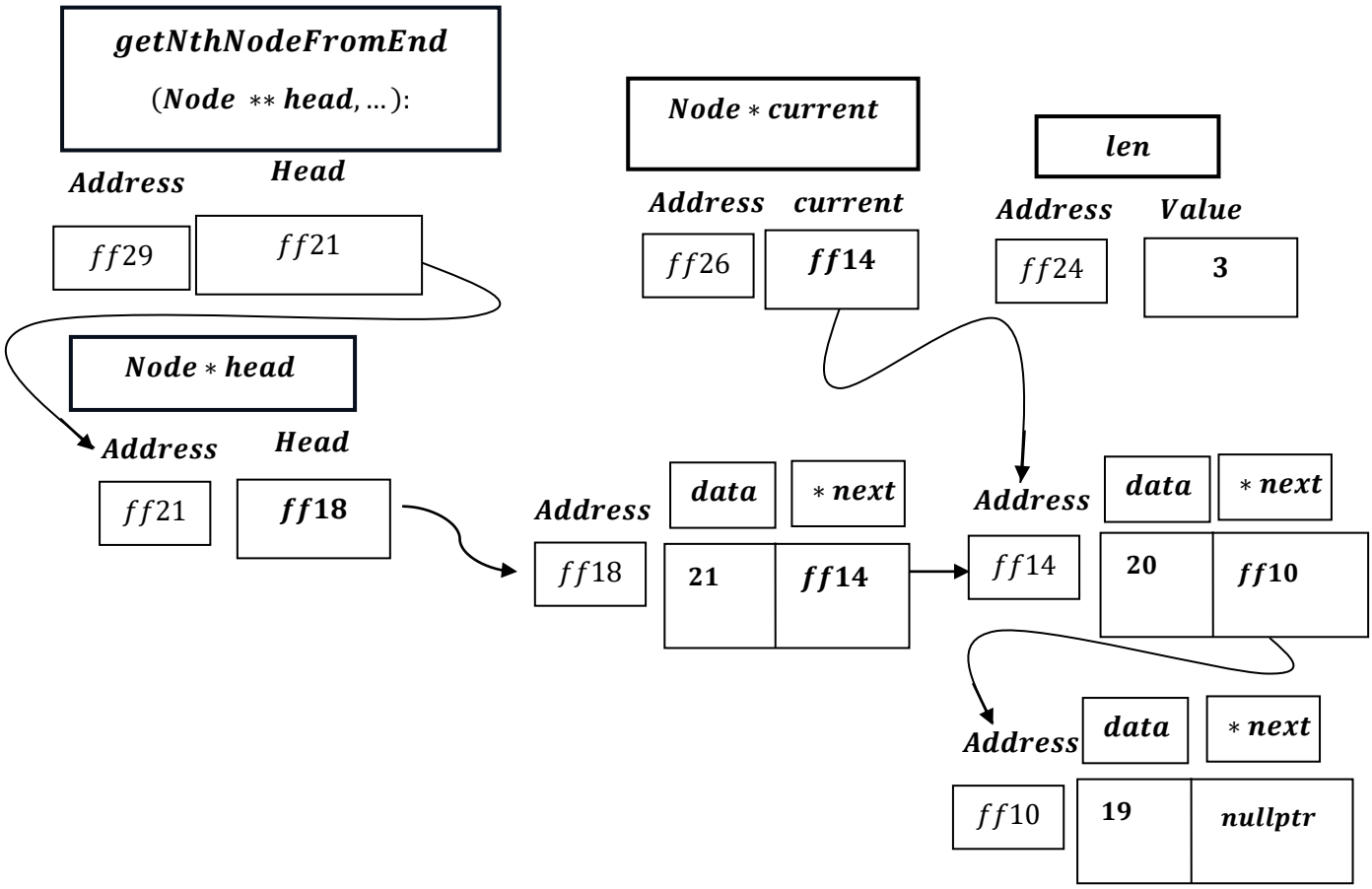
```
for (int i = 0; i < len - node; i++)
        {
            current = current->next;
        }
```

$Say, node = 2(user\ input).$

$for\ i = 0\ and\ i < (len: 3 - node: 2)\ i.e.\ i < 1:$

$current = current \rightarrow next: ff14;$

$i = i + 1 = 0 + 1 = 1;$



$for\ i = 1\ and\ i < (len: 3 - node: 2)\ i.e.\ i < 1, hence\ \ condition\ is\ false\ i.e.$

$(i = 1\ and\ i < 1), hence\ loop\ exits.$

```
        return current->data;
```

$Now, it\ returns\ current \rightarrow data; i.e, data\ stored\ in\ 2nd\ Node\ i.e.\ 20$.

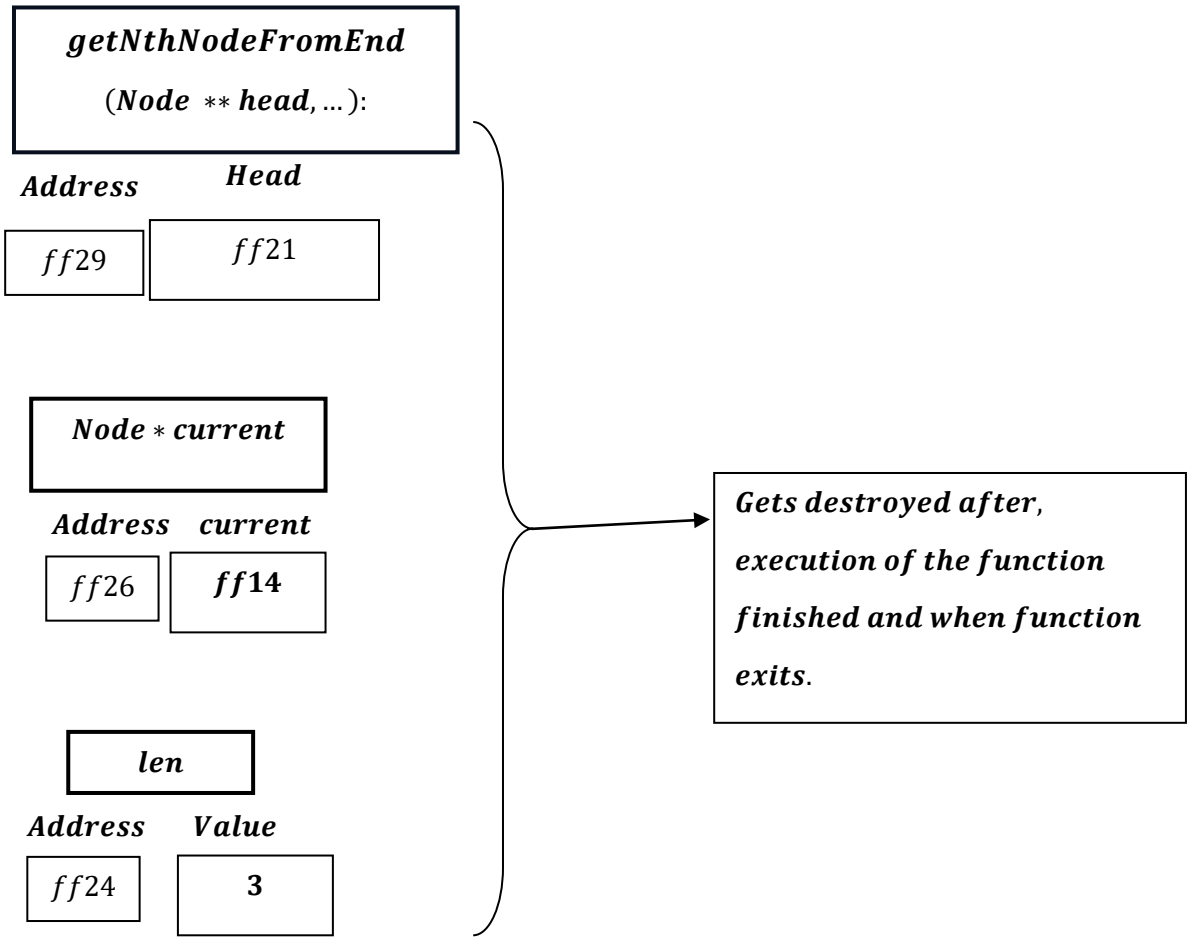
$Thus\ we\ get\ Node\ from\ End\ i.e.\ if\ we\ enter\ node: 1 \rightarrow It\ will\ give\ us\ n^{th}\ node's\ data.$

$if\ we\ enter\ node: 2 \rightarrow It\ will\ give\ us\ (n-1)^{th}\ node's\ data.$

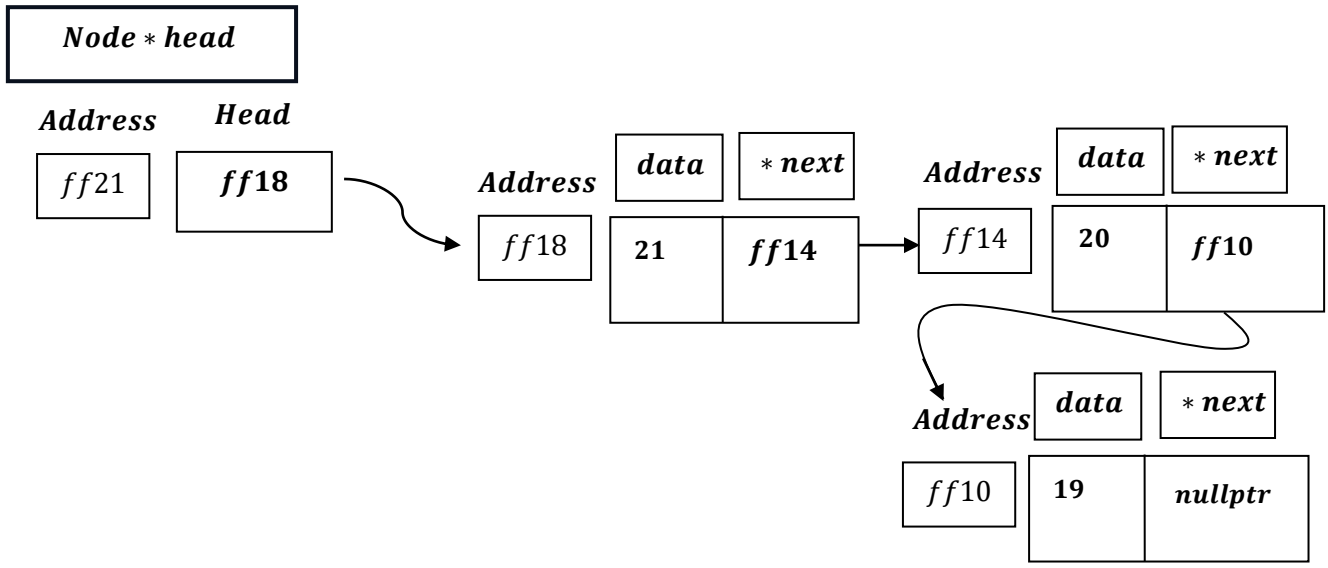
$if\ we\ enter\ node: 3 \rightarrow It\ will\ give\ us\ (n-3)^{th}\ node's\ \ data.$

$\ldots.\ And\ it\ goes\ like\ this.$

$Now, as\ the\ function\ get\ finished, all\ temporary\ varaible\ created\ will\ get\ destroyed.$



And we are left with:



Now, `getNthNodeFromEnd` step by step explanation :

1. *Initialize Length and Current Pointer*:

- ***An integer len is initialized to 0 to keep track of the length of the linked list.***
- ***A pointer current is initialized to point to the head of the list.***

2. *Calculate the Length of the List*:

- ***The function enters a loop to traverse the entire list to calculate its length.***
- ***In each iteration, it moves the current pointer to the next node and increments len.***

3. *Check for Invalid Node Index*:

- ***The function checks if the provided node index is less than 1 or greater than the length of the list.***

- ***If the index is invalid, it prints an error message and returns -1.***

4. Reset Current Pointer:

- *The current pointer is reset to point to the head of the list again.*

5. Traverse to the $(len - node)$th Node:

- *The function enters a loop to traverse the list until it reaches the $(len - node)$th node from the start.*
- *This is equivalent to the nth node from the end of the list.*

6. Return Node Data:

- *Once the desired node is reached, the function returns the data of the current node.*

Time Complexity of getNthNodeFromEnd

```
int getNthNodeFromEnd(Node **head, int node)
{
    int len = 0;                          O(1)
    Node *current;                        O(1)
    current = *head;                      O(1)
    while (current != nullptr)
    {
        current = current->next;          runs n times.
        len++;                            runs n times.
    }
    if (node < 1 || node > len)           checks condition in constant time.
    {
        cout << "Error: Invalid index." << endl;
        return -1;                        runs 1 time i.e. O(1)
    }
```

```
    else
    {
        current = *head;
        for (int i = 0; i < len - node; i++)
        {
            current = current->next;
        }
        return current->data;
    }
}
```

$takes\ O(1)\ time.$

$if\ node\ is\ 1, loop\ runs\ `n-1`\ times,$
$statement\ also\ runs\ `n-1`\ times.$

$takes\ O(1)\ time.$

Worst Case

1. When Node is negative and node is greater than length:

- $when\ `len = 0`\ , this\ assignment\ takes\ constant\ amount\ of\ time\ , O(1).$
- $Node * current\ ; declaration\ takes\ constant\ amount\ of\ time, O(1).$
- $current = * head; assignment\ takes\ constant\ amount\ of\ time\ , O(1).$
- $Calculation\ of\ length\ `len`\ takes\ `n`\ times\ also\ `current = current \rightarrow next`$
  $assignment\ takes\ `n`\ times\ , hence\ it\ takes\ in\ total: n + n = 2n = O(n).$
- $As\ `node`\ is\ less\ than\ 1\ or\ `node`\ is\ greater\ than\ `len`\ , it\ will\ output$
  $a\ statement\ and\ return - 1\ , which\ takes\ constant\ amount\ of\ time$
  $, i.e. O(1).$

  $Hence\ total\ amount\ of\ time\ taken: -O(1) + O(1) + O(1) + O(n) + O(1) = O(n).$

2. When `node` is entered : `1`

- $when\ `len = 0`\ , this\ assignment\ takes\ constant\ amount\ of\ time\ , O(1).$
- $Node * current\ ; declaration\ takes\ constant\ amount\ of\ time, O(1).$
- $current = * head; assignment\ takes\ constant\ amount\ of\ time\ , O(1).$
- $Calculation\ of\ length\ `len`\ takes\ `n`\ times\ also\ `current = current \rightarrow next`$
  $assignment\ takes\ `n`\ times\ , hence\ it\ takes\ in\ total: n + n = 2n = O(n).$

- $Now\ , node\ is\ `1`\ , then\ it\ just\ checks\ that\ `node`\ less\ than\ `1`\ , which\ is\ false$ $this\ takes\ constant\ `c`\ amount\ of\ time\ which\ is\ `O(1)`\ .$

- $As\ `node`\ input\ is\ 1\ , therefore\ statement\ , `current = current \rightarrow next`\ ,$ $assignment\ takes\ `n-1`\ times. As\ current\ already\ points\ to\ head\ pointer$ $or\ 1st\ node\ to\ start\ with\ and\ while\ loop\ starts\ it\ will\ get\ assigned\ to$ $address\ of\ 2nd\ node\ , then\ the\ 3rd\ node, \ldots , to\ address\ of\ nth\ node. Hence$ $2nd\ to\ nth\ node\ i.e. loop\ runs\ n-1\ times.$

  $Therefore\ , this\ results\ , O(n-1) = O(n).$

- $Now, it\ returns\ current \rightarrow data; which\ takes\ constant\ `c`\ amount\ of\ time,$ $which\ is : O(1).$

  $Hence\ total\ amount\ of\ time\ taken: -O(1) + O(1) + O(1) + O(n) + O(1) +$ $O(n) + O(1) = O(n).$

Average Case

$$\left(\begin{array}{c} uniform\ distribution\ of\ nodes - \\ distribution\ where\ all\ outcomes\ are\ equally\ likely. \end{array}\right)$$

$Uniform\ Distribution: This\ is\ a\ statistical\ distribution\ where\ each$ $possible\ value\ of\ a\ random\ variable\ is\ equally\ likely. In\ the\ context\ of$ $this\ function, it\ means\ that\ each\ possible\ value\ of\ node$ $(from\ 1\ to\ the\ length\ of\ the\ list)\ is\ equally\ likely\ to\ be\ requested.$

$Average\ Case: When\ analyzing\ the\ average\ case\ performance\ of$ $an\ algorithm, we\ consider\ the\ expected\ performance\ over\ all\ possible\ inputs,$ $assuming\ a\ certain\ distribution\ of\ inputs. For\ the\ getNthNodeFromEnd$ $function, if\ we\ assume\ a\ uniform\ distribution\ of\ node, it\ means\ that$ $each\ node\ position\ from\ the\ end\ is\ equally\ likely\ to\ be\ requested.$

$Continuing\ with\ the\ calculation:$

$\rightarrow$ $1st\ we\ have\ to\ calculate\ the\ length\ of\ the\ linked\ list\ i.e.`n`, \Theta(n)$.

$\rightarrow$ $2nd\ based\ on\ `node`\ input, where\ `n`\ is\ length\ of\ linked\ list, then$:

$When\ node\ is\ 1, loop\ runs\ `n-1`\ times.$
$When\ node\ is\ 2, loop\ runs\ `n-2`\ times.$
$When\ node\ is\ 3, loop\ runs\ `n-3`\ times.$
.
.
.
$When\ node\ is\ `n-1`, loop\ runs\ 1\ time.$
$When\ node\ is\ `n`, loop\ runs\ 0\ time.$

$Therefore, when\ node\ entered\ from \left[\sum_{i=1}^{n} i\right], loop\ runs\ from \left[\sum_{i=0}^{n-1} i\right]\ times.$

$$\sum_{i=0}^{n-1} i = a_0 + \sum_{i=1}^{n-1} i = 0 + \frac{n(n-1)}{2} = \frac{n(n-1)}{2}.$$

$Now, when\ `node = n`, loop\ runs\ 0\ time\ but\ `current \rightarrow data`\ returns\ i.e.\ the\ first\ node.$

$Hence, we\ have, \frac{n(n-1)}{2} + 1. Here\ 1\ for\ finding\ the\ first\ node.$

$Now, If\ there\ is\ `n`\ no.\ of\ nodes\ there\ we\ find\ data\ at\ `n`\ positions\ equally\ likely\ is$:

$For\ position\ 1, the\ probability\ of\ finding\ data\ is: \frac{1}{n}.$

$For\ position\ 2, the\ probability\ of\ finding\ data\ is: \frac{1}{n}.$

.

.

$For\ position\ n, the\ probability\ of\ finding\ data\ is: \frac{1}{n}.$

$$\boldsymbol{Hence\ probability\ of\ finding\ data\ \ will\ always\ remain}: \frac{\mathbf{1}}{\boldsymbol{n}}\ , \boldsymbol{equally\ likely}.$$

$$\therefore\ \boldsymbol{Average\ Time\ Complexity\ \ is}:\ \boldsymbol{probability\ of\ finding\ data} \times (\boldsymbol{number\ of\ possible\ iterations} + \boldsymbol{constant\ time\ complexities\ for\ left\ over\ positions})\ .$$

$$= \frac{\mathbf{1}}{\boldsymbol{n}} \times \left(\frac{\boldsymbol{n}(\boldsymbol{n}-\mathbf{1})}{\mathbf{2}} + \mathbf{1}\right)$$

$$= \frac{\boldsymbol{n}-\mathbf{1}}{\mathbf{2}} + \frac{\mathbf{1}}{\boldsymbol{n}}$$

$$\boldsymbol{Appplying\ Big\ Theta\ in\ the\ equation}:$$

$$= \Theta\left(\frac{\boldsymbol{n}-\mathbf{1}}{\mathbf{2}}\right) + \Theta\left(\frac{\mathbf{1}}{\boldsymbol{n}}\right)$$

$$\boldsymbol{Now, taking\ out\ the\ dominant\ term\ of\ numerator\ and\ denominator}:$$

$$= \Theta\left(\frac{\boldsymbol{n}}{\mathbf{2}}\right) + \Theta\left(\frac{\mathbf{1}}{\boldsymbol{n}}\right)$$

$$= \frac{\mathbf{1}}{\mathbf{2}} \times \Theta(\boldsymbol{n}) + \Theta\left(\frac{\mathbf{1}}{\boldsymbol{n}}\right)$$

$$\boldsymbol{Now\ removing\ the\ constant}\ `\frac{\mathbf{1}}{\mathbf{2}}`\ \boldsymbol{we\ get}:$$

$$= \Theta(\boldsymbol{n}) + \Theta\left(\frac{\mathbf{1}}{\boldsymbol{n}}\right)$$

$When\ combining\ two\ Big\ Theta\ \Theta$ **terms , largest growth rate is the result**:

$\rightarrow \Theta(n)\ grows\ linearly\ with\ `n`$.

$\rightarrow \Theta\left(\frac{1}{n}\right)\ decreases\ to\ a\ constant\ , i.e.\ when\ `n`\ grows\ larger\ , \frac{1}{n}\ approaches\ to\ 0.$

$This\ we\ can\ prove\ by\ limit\ and\ continuity\ also$:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0\ \left[\frac{c}{\infty} = 0\ , where\ c\ is\ constant\right].$$

$Hence\ we\ get\ `\Theta(n)`\ for\ all\ positions.$

$But\ as\ length\ already\ calculated\ earlier\ i.e.\ \Theta(n), now\ calculation\ for\ all\ possibility\ to\ get\ `n`\ th\ \ positions\ equally\ likely\ doesnot\ actually\ matters,\ hence\ , either\ from\ calculation\ of\ length\ of\ node, we\ can\ say\ directly\ that\ Average\ Case\ is\ \Theta(n), or\ ,$

$Total\ complexity$: $\Theta(n) + \Theta(n) = \Theta(n)$.

$Note\ , by\ relation$: $\Omega \leq \Theta \leq O\ , we\ get$:

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$$\Rightarrow c_1 \times 1 \leq n + \frac{n-1}{2} + \frac{1}{n} \leq c_2 \times n$$

$Alternatively,$

$\rightarrow$ $\boldsymbol{1st\ we\ have\ to\ calculate\ the\ length\ of\ the\ linked\ list\ i.e.`n`\ ,\Theta(n)}$.

$\boldsymbol{For\ position\ 1, the\ probability\ of\ finding\ data\ is: \frac{1}{n}}$.

$\boldsymbol{For\ position\ 2, the\ probability\ of\ finding\ data\ is: \frac{1}{n}}$.

.

.

$\boldsymbol{For\ position\ N, the\ probability\ of\ finding\ data\ is: \frac{1}{n}}$.

$\boldsymbol{Hence\ probability\ of\ finding\ data\ always\ remain\ \frac{1}{n}, for\ position\ 1\ to\ n}$.

$$\boldsymbol{Average\ case\ complexity:\ Probability\ of\ finding\ data\ \times \begin{pmatrix} Summation \\ of\ number\ of \\ Position. \end{pmatrix}}$$

$$= \frac{1}{n} \times \left(\sum_{i=1}^{n} i \right)$$

$$= \frac{1}{n} \times \left(\frac{n(n+1)}{2} \right)$$

$$= \frac{n+1}{2}$$

$$or, \frac{n}{2} + \frac{1}{2}$$

$\boldsymbol{Appplying\ Big\ Theta\ in\ the\ equation}$:

$$= \Theta\left(\frac{n}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

$\boldsymbol{Now\ removing\ the\ constant\ `\frac{1}{2}`\ we\ get}$:

$$= \Theta(n) + \Theta(1)$$

$$= \Theta(n).$$

$\boldsymbol{Total\ complexity}$: $\Theta(n) + \Theta(n) = \Theta(n)$.

$\boldsymbol{Note\ , by\ relation}$: $\Omega \leq \Theta \leq O$, $\boldsymbol{we\ get}$:

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Longrightarrow c_1 \times 1 \leq n + \frac{n+1}{2} \leq c_2 \times n$$

$\boldsymbol{Singly\ Linked\ List}$	$\boldsymbol{Cases}$	$\boldsymbol{Time\ Complexity}$
$\boldsymbol{Get\ Nth\ Node\ From\ End}$	$\boldsymbol{1.\ Best\ Case}$	$\Omega(n)$ [$\boldsymbol{As, length\ is\ calculated\ first\ hence\ Worst\ Case = Best\ Case\ , here}$]
	$\boldsymbol{2.\ Worst\ Case}$	$O(n)$
	$\boldsymbol{3.\ Average\ Case}$	$\Theta(n)$

****